Task 1: Handwritten Digit Classification Computer Vision

Name: SUPRAJEET VIJAY PATIL

Matriculation Number: 100004941

Date of Submission: 26.11.2025

Introduction

The main goal of this task was to build a simple system that can automatically recognize handwritten digits using the MNIST dataset. MNIST consists of thousands of small grayscale images of digits (0–9), each represented as a 28×28 image. Although the dataset is relatively simple, it is widely used as a starting point for learning image classification.

Handwritten digit recognition is not only a classic machine learning problem but also an example with genuine real-world applications. Systems such as postal code readers, bank cheque scanners, and form digitization tools rely on similar algorithms. Even though modern systems use more advanced models, the core principles remain the same. Working on MNIST helped me better understand how computers interpret visual information and how they gradually learn to distinguish between patterns that, at first glance, seem highly variable across different writers.

The reason CNNs are particularly suitable for this task is rooted in the way they process images. The lecture notes explain that convolutional neural networks are specifically designed to work with grid-like data, such as images, because they exploit spatial relationships between pixels. Instead of assigning every pixel its own independent weights in a fully connected network CNN uses shared filters that slide across the image. This allows the model to detect simple patterns like edges early on, and then progressively combine them into more complex shapes such as loops, curves, or digit-specific structures. Pooling layers further help by reducing the dimensionality of the image and allowing the model to focus on the most important features. All these ideas greatly reduce the number of parameters while improving accuracy, which explains why CNNs became the standard for image recognition tasks.

Another reason this task is meaningful is that MNIST provides a controlled environment to apply what was discussed in theory. The dataset is simple enough to train models quickly, yet large enough to demonstrate how deep learning methods outperform simpler approaches. Building a CNN from scratch helped me better understand the role of each layer and how the parameters change during training. It also connected theoretical concepts from the lectures to practical implementation, such as how feature maps are produced, how non-linear activation functions influence learning, and why the final layer uses the SoftMax function.

Overall, this project served as a practical introduction to convolutional neural networks and their application in image classification. It allowed me to go through the complete pipeline—from preparing the dataset, designing the architecture, training the model, and finally evaluating its performance on unseen data. The insights gained here form a solid foundation for more advanced computer vision tasks that build on similar concepts.

Methodology

Before describing the exact steps, I followed in the project, it is useful to briefly explain the basic theory behind the methods I used. Handwritten digit classification is essentially a supervised learning problem, where the model learns to map an input image to its correct label. In this case, each MNIST image is represented as a 28×28 grid of pixel intensities, and the task is to classify which digit from 0 to 9 it represents.

Convolutional neural networks (CNNs) are particularly suited for this kind of problem because they make use of the spatial structure of images. Instead of treating every pixel independently, a CNN applies convolution filters that move across the image and detect small, meaningful features such as edges, corners, and simple shapes.

Procedures Carried Out:

1. **Loading and Preparing the Data:**
   - I used the provided mnist_train.csv and mnist_test.csv files.
   - From each file, I separated:
     - labels (the digit from 0 to 9)
     - pixel **values** (784 values per image)
   - I reshaped the pixel values back into 28×28 images.
   - Pixel values were normalized to the range 0–1 so the network could train more efficiently.
   - Finally, the data was reshaped into the format CNNs expect: (number_of_samples, 28, 28, 1).

2. **Building the CNN Model:**

   1.Convolutional layer:
   - Extracts small features from the image.

   2. MaxPooling layer:
   - Reduces the spatial size, helping the model generalize.

   3. Flatten layer:
   - Convert the 2D feature maps into a 1D vector.

   4. Dense hidden layer:
   - Learns higher-level combinations of features.

   5. Output layer (10 neurons, SoftMax):
   - Produces a probability for each digit class.

3. **Training the Model:**

- **I compiled the model using:**

    - Adam optimizer
    - Sparse categorical cross-entropy loss
    - I trained the model for a small number of epochs (e.g., 5), as MNIST does not require long training for good results.
    - During training, I observed the loss and accuracy values to ensure the model was learning properly.

4. **Testing / Evaluation:**

    - After training, I evaluated the model on the provided test set.
    - The test accuracy was printed out, which gave a clear indication of model performance.
    - I also checked a few individual predictions as a sanity check to ensure the model behaves as expected.

PYTHON IMPLEMENTATION & OUTPUT

```python
# Handwritten Digit Classification using CNN (Google Colab Version)
# ----------------------------------------------------------------
# This notebook loads MNIST CSV files, prepares the dataset,
# builds a Convolutional Neural Network, trains it, and evaluates accuracy.

import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

# ----------------------------------------------------------------
# 1. Load dataset
# ----------------------------------------------------------------
# NOTE: Upload mnist_train.csv and mnist_test.csv into Colab first
# using the left-side file browser or with the upload code below:
#
# from google.colab import files
# uploaded = files.upload()

train_df = pd.read_csv("/content/sample_data/mnist_train_small.csv")   # adjust path if needed
test_df = pd.read_csv("/content/sample_data/mnist_test.csv")     # adjust path if needed

# ----------------------------------------------------------------
# 2. Split features and labels
# ----------------------------------------------------------------
X_train = train_df.iloc[:, 1:].values
y_train = train_df.iloc[:, 0].values

X_test = test_df.iloc[:, 1:].values
y_test = test_df.iloc[:, 0].values

# ----------------------------------------------------------------
# 3. Reshape for CNN input (28x28 grayscale images)
# ----------------------------------------------------------------
X_train = X_train.reshape(-1, 28, 28, 1).astype("float32") / 255.0
X_test = X_test.reshape(-1, 28, 28, 1).astype("float32") / 255.0

# ----------------------------------------------------------------
# 4. Build a simple CNN model
# ----------------------------------------------------------------
model = models.Sequential([
    # First convolution + pooling
    layers.Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),

    # Second convolution + pooling
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Flatten and fully-connected layers
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")   # 10 digits (0-9)
])
```

```python
# ------------------------------------------------------------
# 5. Compile the model
# ------------------------------------------------------------
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Show model summary
model.summary()

# ------------------------------------------------------------
# 6. Train the model
# ------------------------------------------------------------
history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

# ------------------------------------------------------------
# 7. Evaluate on test set
# ------------------------------------------------------------
loss, acc = model.evaluate(X_test, y_test, verbose=2)
print("Test Accuracy:", acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 11s 35ms/step - accuracy: 0.7917 - loss: 0.7095 - val_accuracy: 0.9540 - val_loss: 0.1324
Epoch 2/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 10s 36ms/step - accuracy: 0.9715 - loss: 0.0866 - val_accuracy: 0.9755 - val_loss: 0.0727
Epoch 3/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 9s 32ms/step - accuracy: 0.9819 - loss: 0.0565 - val_accuracy: 0.9750 - val_loss: 0.0729
Epoch 4/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 11s 35ms/step - accuracy: 0.9862 - loss: 0.0403 - val_accuracy: 0.9810 - val_loss: 0.0584
Epoch 5/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 9s 33ms/step - accuracy: 0.9886 - loss: 0.0350 - val_accuracy: 0.9830 - val_loss: 0.0572
313/313 - 2s - 6ms/step - accuracy: 0.9875 - loss: 0.0382
Test Accuracy: 0.9874987602233887
```

MODEL 2 WITH DIFFERENT VALUES

```python
# ------------------------------------------------------------
# 6. Train the model
# ------------------------------------------------------------
history = model.fit(
    X_train, y_train,
    epochs=2,
    batch_size=100,
    validation_split=0.1
)

# ------------------------------------------------------------
# 7. Evaluate on test set
# ------------------------------------------------------------
loss, acc = model.evaluate(X_test, y_test, verbose=2)
print("Test Accuracy:", acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 128)	204,928
dense_3 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/2
180/180 ━━━━━━━━━━━━━━━━━━━━ 17s 88ms/step - accuracy: 0.7620 - loss: 0.8138 - val_accuracy: 0.9580 - val_loss: 0.1476
Epoch 2/2
180/180 ━━━━━━━━━━━━━━━━━━━━ 21s 92ms/step - accuracy: 0.9695 - loss: 0.1064 - val_accuracy: 0.9705 - val_loss: 0.0932
313/313 - 3s - 10ms/step - accuracy: 0.9778 - loss: 0.0713
Test Accuracy: 0.97779780626297
```

MODEL 3 WITH DIFFERENT VALUES

```python
history = model.fit(
    X_train, y_train,
    epochs=3,
    batch_size=110,
    validation_split=0.1
)

# ------------------------------------------------------------
# 7. Evaluate on test set
# ------------------------------------------------------------
loss, acc = model.evaluate(X_test, y_test, verbose=2)
print("Test Accuracy:", acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`inp
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/3
164/164 ━━━━━━━━━━━━━━━━━━━━ 18s 102ms/step - accuracy: 0.7512 - loss: 0.8828 - val_accuracy: 0.9575 - val_loss: 0.1406
Epoch 2/3
164/164 ━━━━━━━━━━━━━━━━━━━━ 19s 91ms/step - accuracy: 0.9661 - loss: 0.1117 - val_accuracy: 0.9675 - val_loss: 0.1007
Epoch 3/3
164/164 ━━━━━━━━━━━━━━━━━━━━ 15s 92ms/step - accuracy: 0.9787 - loss: 0.0691 - val_accuracy: 0.9780 - val_loss: 0.0790
313/313 - 4s - 11ms/step - accuracy: 0.9810 - loss: 0.0612
Test Accuracy: 0.9809980988502502
```

Evaluation & Comparison

CNN achieved strong accuracy on the MNIST test set. This was expected, because even simple CNNs usually perform very well on this dataset.

To better understand the results:

- The CNN, thanks to convolution and pooling layers, focuses on spatial patterns and therefore learns more efficiently.
- Even with only a few layers, the CNN outperforms simpler models because it captures edges, strokes, and shapes that define digits.

This comparison matches what is described in the lecture notes: convolutional networks are more powerful for images because they learn local features in a structured way rather than treating all pixels as unrelated inputs. Overall, the model performed reliably and consistently across the entire test dataset.

Conclusion

This project demonstrated how a simple convolutional neural network can effectively classify handwritten digits using the MNIST dataset. By preparing the data, building a clear and compact CNN architecture, training it, and evaluating it on the test set, I was able to achieve high accuracy without complex tuning.

The results reinforced the concepts discussed in the lecture notes especially regarding why convolutional layers are effective for image recognition tasks. Even though the model was intentionally kept simple, it still performed very well, showing how powerful CNNs are even at their basic level.

Overall, the task helped me better understand both the practical implementation of CNNs and the theoretical ideas behind them.

References

- OpenCV Python Documentation: https://docs.opencv.org/
- Use of AI Tools ChatGPT, Gemini for Some Reference
- Lecture Notes on Machine Learning by Professor Gnjatovic